RISC-V 操作数

名字	示例	注解
32个寄存器	x0～x31	快速定位数据。在RISC-V中，只对在寄存器中的数据执行算术运算
2^{61}个存储字	Memory[0], Memory[8], ..., Memory[18 446 744 073 709 551 608]	只能被数据传输指令访问。RISC-V使用字节寻址，因此顺序双字访问相差8。存储器保存数据结构、数组和换出的寄存器的内容

RISC-V 汇编语言

类别	指令	示例	含义	注解
算术运算	加	add x5, x6, x7	x5 = x6 + x7	三寄存器操作数；加
	减	sub x5, x6, x7	x5 = x6 - x7	三寄存器操作数；减
	立即数加	addi x5, x6, 20	x5 = x6 + 20	用于加常数
数据传输	取双字	ld x5, 40(x6)	x5 = Memory[x6 + 40]	从存储器取双字到寄存器
	存双字	sd x5, 40(x6)	Memory[x6 + 40] = x5	从寄存器存双字到存储器
	取字	lw x5, 40(x6)	x5 = Memory[x6 + 40]	从存储器取字到寄存器
	取字（无符号数）	lwu x5, 40(x6)	x5 = Memory[x6 + 40]	从存储器取无符号字到寄存器
	存字	sw x5, 40(x6)	Memory[x6 + 40] = x5	从寄存器存字到存储器
	取半字	lh x5, 40(x6)	x5 = Memory[x6 + 40]	从存储器取半字到寄存器
	取半字（无符号数）	lhu x5, 40(x6)	x5 = Memory[x6 + 40]	从存储器取无符号半字到寄存器

RISC-V 汇编语言

类别	指令	示例	含义	注解
数据传输	存半字	sh x5, 40(x6)	Memory[x6 + 40] = x5	从寄存器存半字到存储器
	取字节	lb x5, 40(x6)	x5 = Memory[x6 + 40]	从存储器取字节到寄存器
	取字节（无符号数）	lbu x5, 40(x6)	x5 = Memory[x6 + 40]	从存储器取无符号字节到寄存器
	存字节	sb x5, 40(x6)	Memory[x6 + 40] = x5	从寄存器存字节到存储器
	取保留字	lr.d x5, (x6)	x5 = Memory[x6]	取；原子交换的前半部分
	存条件字	sc.d x7, x5, (x6)	Memory[x6] = x5; x7 = 0/1	存；原子交换的后半部分
	取立即数高位	lui x5, 0x12345	x5 = 0x12345000	取左移12位后的20位立即数
逻辑运算	与	and x5, x6, x7	x5 = x6 & x7	三寄存器操作数；按位与
	或	or x5, x6, x8	x5 = x6 | x8	三寄存器操作数；按位或
	异或	xor x5, x6, x9	x5 = x6 ^ x9	三寄存器操作数；按位异或
	与立即数	andi x5, x6, 20	x5 = x6 & 20	寄存器与常数按位与
	或立即数	ori x5, x6, 20	x5 = x6 | 20	寄存器与常数按位或
	异或立即数	xori x5, x6, 20	x5 = x6 ^ 20	寄存器与常数按位异或
移位操作	逻辑左移	sll x5, x6, x7	x5 = x6 << x7	按寄存器给定位数左移
	逻辑右移	srl x5, x6, x7	x5 = x6 >> x7	按寄存器给定位数右移
	算术右移	sra x5, x6, x7	x5 = x6 >> x7	按寄存器给定位数算术右移
	逻辑左移立即数	slli x5, x6, 3	x5 = x6 << 3	根据立即数给定位数左移
	逻辑右移立即数	srli x5, x6, 3	x5 = x6 >> 3	根据立即数给定位数右移
	算术右移立即数	srai x5, x6, 3	x5 = x6 >> 3	根据立即数给定位数算术右移
条件分支	相等即跳转	beq x5, x6, 100	if (x5 == x6) go to PC+100	若寄存器数值相等则跳转到PC相对地址
	不等即跳转	bne x5, x6, 100	if (x5 != x6) go to PC+100	若寄存器数值不等则跳转到PC相对地址
	小于即跳转	blt x5, x6, 100	if (x5 < x6) go to PC+100	若寄存器数值比较结果小于则跳转到PC相对地址
	大于等于即跳转	bge x5, x6, 100	if (x5 >= x6) go to PC+100	若寄存器数值比较结果大于或等于则跳转到PC相对地址
	小于即跳转（无符号）	bltu x5, x6, 100	if (x5 < x6) go to PC+100	若寄存器数值比较结果小于则跳转到PC相对地址（无符号）
	大于等于即跳转（无符号）	bgeu x5, x6, 100	if (x5 >= x6) go to PC+100	若寄存器数值比较结果大于或等于则跳转到PC相对地址（无符号）
无条件跳转	跳转-链接	jal x1, 100	x1 = PC+4; go to PC+100	用于PC相关的过程调用
	跳转-链接（寄存器地址）	jalr x1, 100(x5)	x1 = PC+4; go to x5+100	用于过程返回；非直接调用

> RSIC-V算法指令只执行一个操作，并且必须总是只有三个变量

RISC-V概述

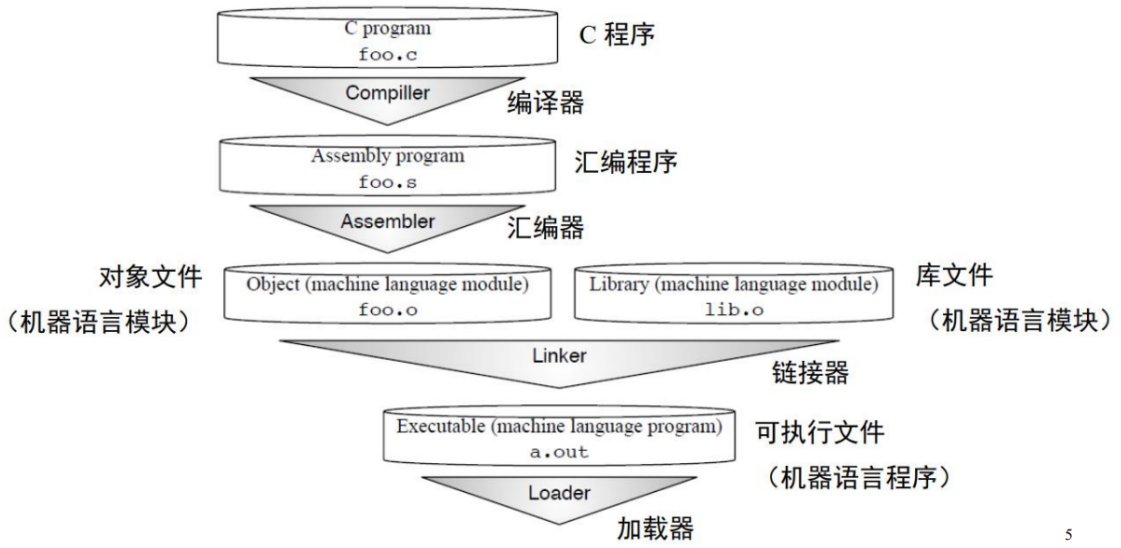
指令系统基本概念

主流指令集及发展方向

RISC-V指令集

RISC-V汇编语言

- 汇编语言（Assembly Language）是一种“低级”语言， 直接接触最底层的硬件，与底层的CPU指令集系统架构有关，需要对底层硬件非常熟悉才能编写出高效的汇编程序。在汇编语言中，用**助记符**（一些容易理解和记忆的字母、 单词）代替机器指令的操作码，用**地址符号或标号**代替指令或操作数的地址。
- C语言程序转换到机器语言程序共需要四步（注：不一定严格按照这个四个步骤执行）：
  - 将高级语言程序编译成汇编语言程序
  - 用机器语言组装成目标模块
  - 链接器将多个模块与库程序组合在一起以解析所有引用
  - 加载器将机器代码放入适当的存储器位置以供处理器执行



- **编译器：**将高级语言程序转换为机器能理解的符号形式（汇编语言程序）
- **汇编器：**将汇编语言转换为目标文件，该目标文件是机器指令、数据和将指令正确放入内存所需信息的组合
  - 汇编器还可以处理机器指令的常见变体，这类指令称为**伪指令**

    例如：li x9, 123　　汇编器将其转化为 addi x9, x0, 123
- 链接器：链接器将所有独立汇编的机器语言程序“缝合” 在一起。由于独立编译和汇编每个过程，因此更改一行代码只需要编译和汇编一个过程，链接器有用的原因是 修正代码要比重新编译和重新汇编快得多。
- 加载器：将可执行程序放在主存中以准备执行的系统程序。
- 动态链接库(Dynamic Link Library)：是在执行期间链接到程序的库例程。DLL需要额外的空间来存储动态链接所需的信息，但不要求复制或链接整个库。在第一次调用例程时会付出大量的开销，但此后只需一个间接跳转。
- 汇编语言的优缺点：

    - 缺点：难读、难写、难移植
    - 优点：灵活、强大
- 汇编语言的应用场景
    - 需要直接访问底层硬件
    - 需要对性能进行优化

RISC-V汇编指令概览

- **通用指令格式**

$$op \quad dst, src1, src2$$

  1个操作码，3个操作数

  op（operation）：操作的助记符

  dst（destination）：目标寄存器

  src1（source1）：第一个源操作数寄存器

  src2（source2）：第二个源操作数寄存器

- 汇编格式:
    - 每一行最多一条指令
    - 每一条指令只有一个操作
    - 汇编语言可以实现C语言中类似操作（= + - * / &|）

      C语言中的操作会被分解为一条或者多条汇编指令

      C语言中的一行会被编译为多行RISC-V汇编程序
- RISC-V汇编指令操作对象
    - 寄存器：

      32个通用寄存器，x0 ~ x31（注意：仅涉及 RV32I 的通用寄存器组）

      在 RISC-V 中，算术逻辑运算所操作的数据必须直接来自寄存器

      x0是一个特殊的寄存器，只用于全零

      每一个寄存器都有别名便于使用，实际硬件并没有任何区别

      RV32I指令集通用寄存器是32位的；RV64I指令集的通用寄存器是64位
    - 内存：

      可以执行在寄存器和内存之间的数据读写操作

      读写操作使用**字节（Byte）**为基本单位进行寻址

      RV64可以访问最多 **2^{64}个字节的内存空间**，即**2^{61}个存储单元**
- **汇编语言的变量：寄存器**
    - 汇编语言不能使用变量（C、JAVA可以）
    - 寄存器变量没有数据类型
    - 好处：寄存器是最快的数据存储单元

      缺陷：寄存器数量有限，需要充分和高效地使用各寄存器

- 32个RISC-V寄存器：

寄存器	助记符	备注
x0	zero	固定值为0
x1	ra	返回地址(Return Address)
x2	sp	栈指针(Stack Pointer)
x3	gp	全局指针(Global Pointer)
x4	tp	线程指针(Thread Pointer)
x5-x7	t0-t2	临时寄存器（temporary register）
x8	s0/fp	save寄存器/帧指针(Frame Pointer)
x9	s1	save寄存器（Save Register）
x10-x11	a0-a1	过程参数 / 过程返回值(Return Value)
x12-x17	a2-a7	过程参数(function argument)
x18-x27	s2-s11	save寄存器
x28-x31	t3-t6	临时寄存器

> 在 RISC-V体系结构中，寄存器的大小为64位；成组的64位频繁出现，因此它们在RISC-V体系结构中被命名为**双字**（计算机中一种访问基本单位，通常是64位一组；对应于RISC-V体系结构中寄存器的大小）。另一个常见大小是成组的32位，在RISC-V体系结构中称为**字**（计算机中另一种访问基本单位，通常是32位一组）。

RISC-V常用汇编指令

> RSIC-V实现的时候用**补码**表示

- **特殊的寄存器x0**

  x0是一个特殊的寄存器，值恒为0，且不能被改变（注意：在任意指令中，如果使用x0作为目标寄存器，**将没有任何效果**，仍然保持0不变）。例如：

  - `add x3, x0, x0` # x3=0
  - `add x1, x2, x0` # x1=x2
  - `add x0, x0, x0` # nop

- **立即数**（即数值常数）

  立即数指令的语法：`opi dst, src, imm`

  和从存储器中取出常熟相比，立即数操作速度更快，能耗更低

  操作码的最后一个字符为i的，会将**第二个操作数**认为是一个立即数 （经常用后缀来指明操作数的类型，例如无符号数unsigned的后缀为u）。例如：

  - `addi rd, rs1, immediate` # rd=rs1 + sext(immediate) : # 符号扩展立即数
  - `addi x1, x2, 5` # x1=x2+5
  - `addi x3, x3, 1` # x3=x3+1 （自增1）

  > 注意：在RISC-V中没有立即数减法，因为立即数减法可以转变为加法。这也印证了为什么RISC-V的指令集如此简洁：**如果一个操作可以分解成一个更简单的操作，不要包含它。**
  >
  > - 例如：`addi x3, x4, -10 (in RISC)` # f = g - 10 (in C)

> 但是在RISC-V中有减法的存在，这是因为如果没有减法存在的话，则需要额外的一条指令求出负数的补码，从而实现将减法变成加法。

算术运算

`add、sub、addi、mul、div`

- **加法：add**

  语法：`add  rd, rs1, rs2` （rs1,rs2 为x0-x31中的某一个寄存器）

  功能：把寄存器rs2的值与寄存器 rs1的的值相加，结果写入寄存器rd（忽略算术溢出），记为rd=rs1+rs2

- **减法：sub**

  语法：`sub  rd, rs1, rs2`

  功能：rd=rs1 - rs2 （忽略算术溢出）

> 溢出是因为计算机中表达数本身是有范围限制的，计算的结果没有足够多的位数进行表达
>
> RISC-V 忽略溢出问题，**高位被截断，低位写入目标寄存器**

算术运算举例：假设a->x1, b->x2, c->x3, d->x4, e->x5，将C语言程序"a = (b + c) – (d + e）"编译成RISC-V汇编指令。

```
add x6, x4, x5          # tmp1 = d + e
add x7, x2, x3          # tmp2 = b + c
sub x1, x7, x6          # a = tmp2-tmp1=(b + c) – (d + e)
```

> 指令中可以看到：
>
> 1）如何使用临时寄存器（x5-x7的助记符为t0-t2）
>
> 2）一个简单的C语言表达式可能需要多条汇编语句
>
> 3）# 符号后面是程序的注释

- **乘法：mul**

  语法：`mul  rd, rs1, rs2`

  功能：rd = rs1 × rs2 把寄存器rs2的值乘以寄存器rs1的值， 结果写入寄存器rd，忽略算术溢出。

  - RSIC-V中的乘法忽略算数溢出，但积的长度是乘数和被乘数长度的和，两个32位数的积是64位（除非两个32位的数都特别小）。因为，在RISC-V中有四种乘法指令，可以得到有符号或无符号的64位积：

    1）要得到整数32位乘积（64位中的低32位）就用**mul指令**（低位永远不可能包含符号位，所以无需考虑有符号数和无符号数）

    2）要得到高32位，如果操作数都是有符号数，就用**mulh指令**

    3）如果操作数都是无符号数，就用**mulhu指令**

    4）如果一个有符号一个无符号，可以用**mulhsu指令**

31 25	24 20	19 15	14 12	11 7	6 0	
0000001	rs2	rs1	000	rd	0110011	R mul
0000001	rs2	rs1	001	rd	0110011	R mulh
0000001	rs2	rs1	010	rd	0110011	R mulhsu
0000001	rs2	rs1	011	rd	0110011	R mulhu

- 如果需要获得完整的64位积，建议的指令序列为：

  `mulh[[s]u] rdh, rs1, rs2`（s/u可选）

  `mul rdl, rs1, rs2`

- 源寄存器必须使用相同的顺序，rdh要注意和rs1和rs2都不相同。这样底层的微体系结构就会把两条指令合并成一次乘法操作，而不是两次乘法操作

- **除法：div**

  语法：`div  rd, rs1, rs2`

  功能：rd = rs1 ÷ rs2 从寄存器rs1的值除以寄存器rs2的值，**向零舍入（商取离0近的值，比如-8/3取-2而不取-3）**，将这些数视为二进制补码，商写入寄存器rd

- **取余数：rem**

  语法：`rem  rd, rs1, rs2`

逻辑运算

`and、or、xor、andi、ori、xori`

- 逻辑运算指令：

语法	功能	描述
and rd, rs1, rs2	rd = rs1 & rs2	按位“与”操作，结果写入寄存器rd
andi rd, rs1, imm	rd = rs1 & sext(imm)	把符号位扩展的立即数和寄存器rs1的值进行按位与，结果写入寄存器rd
or rd, rs1, rs2	rd = rs1 | rs2	按位“或”操作，结果写入寄存器rd
ori rd, rs1, imm	rd = rs1 | sext(imm)	把符号位扩展的立即数和寄存器rs1的值进行按位或，结果写入寄存器rd
xor rd, rs1, rs2	rd = rs1 ^ rs2	按位“异或”操作，结果写入寄存器rd
xori rd, rs1, imm	rd = rs1 ^ sext(imm)	把符号位扩展的立即数与寄存器rs1的值按位“异或”，结果写入寄存器rd

- 位操作指令实例：假设: a→x1, b→x2, c→x

指令	C	RSC-V
And	`a = b & c;`	and x1, x2, x3
And Immediate	`a = b & 8;`	andi x1, x2, 8
Or	`a = b | c;`	or x1, x2, x3
Or Immediate	`a = b | 22;`	ori x1, x2, 22
eXclusive OR	`a = b ^ c;`	xor x1, x2, x3
eXclusive OR Immediate	`a = b ^ -5;`	xori x1, x2, -5

- 按位逻辑运算

- 寄存器操作数：`and x5, x6, x7`　# x5 = x6 & x7
- 立即数操作数：`andi x5, x6, 3`　# x5 = x6 & 3

> **RISC-V中没有NOT**：因为一个数与1异或等于这个数的非，所以由于异或的存在，可以实现非运算，以达到RSIC-V的指令集最精简的目的
>
> - 例如：`xori  x5, x6, -1`　　# x5= x6'
>
> 由于-1的补码为11111111_2，**所以任何数与-1异或等于这个数的非。**

移位运算

`sll、srl、sra、slli、srli、srai`

- 移位指令：

Instruction Name	RISC-V
Shift Left Logical	`sll  rd, rs1, rs2`
Shift Left Logical Immediate	`slli rd, rs1, shamt`
Shift Right Logical	`srl  rd, rs1, rs2`
Shift Right Logical Immediate	`srli rd, rs1, shamt`
Shift Right Arithmetic	`sra  rd, rs1, rs2`
Shift Right Arithmetic Immediate	`srai rd, rs1, shamt`

  slli, srli, srai只需要最多移动63位 (对于RV64)，只会使用 immediate低6位的值 （I类型指令）

- 移位运算举例：`sll, slli, srl`

  `slli x11, x12, 2`　# x11=x12<<2

- 算术移位指令：`sra, srai`

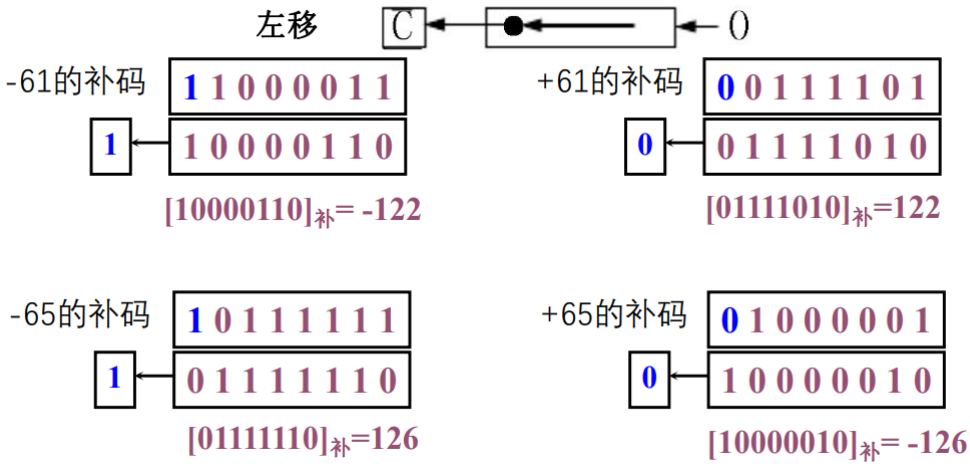
  `srai x10, x10, 4`　# x10=x10>>4

> **RISC-V中没有算术左移指令**：
>
> 我们以8位二进制数的补码移位为例：

真值	补码	逻辑左移	算术左移	结果	逻辑右移1位	算术右移1位
127	01111111	11111110	01111110	错误	00111111	同逻辑右移
.....	.....	.....	.....	错误	.....	.....
64	01000000	10000000	00000000	错误	00100000	同逻辑右移
63	00111111	01111110	01111110	正确	00011111	同逻辑右移
.....	.....	.....	.....	正确	.....	.....
1	00000001	00000010	00000010	正确	00000000	同逻辑右移
-1	11111111	11111110	11111110	正确	01111111	11111111
.....	.....	.....	.....	正确	0xx.....x	11xx.....x
-64	11000000	10000000	10000000	正确	01100000	11100000
-65	10111111	01111110	11111110	错误	01011111	11011111
-66	10111110	01111100	11111100	错误	01011111	11011111
-128	10000000	00000000	10000000	错误	01000000	11000000

> 可以发现，对于右移来说，算数右移和逻辑右移在正确范围内得到的结果是不同的；但是对于左移来说，算数左移和逻辑左移在正确范围内得到的结果是相同的，尽管错误的结果并不一致。所以在正确的范围内，左移就用一套硬件实现就可以，异常处理是另外的事情。

- 带进位的移位：



左移后，如果“进位”和“首位”不一致，则意味着溢出，在硬件上用异或判断即可。

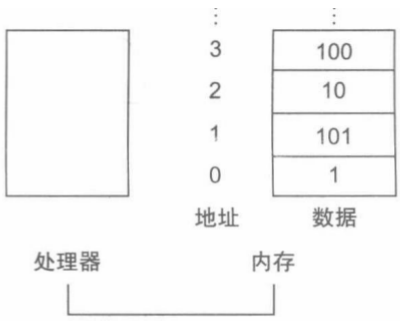
数据传输

`ld、sd、lw、sw、lwu、lh、lhu、sh、lb、lbu、sb、lui`

- 关于数据传输指令的介绍：

  程序语言中，有包含单个数据元素的简单变量，也有更复杂的数据结构——数组和结构体。这些复杂数据结构可以包含比计算机中寄存器数量更多的数据元素。在计算机的五个组成部分中，处理器只能在寄存器中保存少量数据（如C语言中的变量），但是计算机内存可以存储数十亿数据元素。因此，其他的数据结构（如数组和结构体）可以保存在内存中。

  如上所述，除数据传输指令外的RISC-V指令（如算数指令）**只在寄存器中操作**。因此，RISC-V必须包含**在内存和寄存器之间传输数据的指令**，这些指令称为**数据传输指令**。要访问内存中的字或双字，指令必须提供**内存地址**。内存（主存）可以看作一个大型一维数组，其地址作为该数组的下标，从0开始。例如，在下图中，第三个数据元素的地址是2，内存第2号单元存放的数据是10。



内存地址和地址中的内容。如果这些元素是双字，这些地址将是不正确的，因为 RISC-V 实际上使用字节寻址，每个双字代表 8 个字节。图 2-3 展示了顺序双字编址的正确内存寻址

- 内存按**字节**寻址，文件系统在外存上

- 将数据从内存复制到寄存器的数据传输指令通常称为**载入指令**（load）。载入指令的格式是操作名称后面紧跟数据待取的寄存器，然后是寄存器和用于访问内存的常量。指令的常量部分和第二个寄存器中的内容R相加组成内存地址。实际的RISC-V指令名称是ld，表示取双字。
  - Store指令：数据从寄存器到内存
  - Load指令：数据从内存到寄存
- **数据传输指令的格式：** `memop reg, offset(bAddrReg)`
  - memop：操作的名字（load或者store的各种变形）
  - reg：源寄存器或者目标寄存器
  - bAddrReg：保存数据地址的**基址寄存器** (base address register)
  - offset：地址**偏移量**（字节寻址），为立即数 (immediate)
  - 访问的主存地址为 bAddrReg + offset
- 内存的字节寻址方式**（内存是按照"字节"进行编址，不是按照字进行编址的）**

> **存储器寻址**:
>
> 字的定义是由体系结构决定的，不同的体系结构对字（word）的定义不同，例如16位字（Intel、X86）、32位字（MIPS、RISC-V）；如何读多个字节有多种方式，例如32位字：每次一个字节，四次完成/每次一个字，一次完成。
>
> **两个问题：**
>
> 1. **如何将字节地址映射到字地址 (尾端问题)**
>
> 计算机分为两种，一种使用最左边或"大端"字节的地址作为双字地址，另一种使用最右端或"小端"字节的地址作为双字地址。**RISC-V属于后者，称为小端编址。**由于仅在以双字形式和八个单独字节访问相同数据时，字节顺序才会有影响，因此大多数情况下不需要关心"大小端"。
>
> 大端：字地址等于最高字节的地址（字内高字节在最低的地址）
>
> 小端：字地址等于最低字节的地址（字内低字节在最低的地址）
>
> 在一个（双）字内部的字节顺序问题
>
> 高 低
>
> 例如：设地址addr存储的字为 0x89ABCDEF，addr，addr+1，addr+2，addr+3四个字节存放的分别是什么数据？
>
> 地址
>
> | | addr+3 | addr+2 | addr+1 | addr |
> |---|---|---|---|---|
> | 小端(字地址为低字节地址) | 89 | AB | CD | EF |
> | 大端(字地址为高字节地址) | EF | CD | AB | 89 |
>
> 大端(MIPS, IBM360...)
> 小端(Intel 80x86, RISC V...)
>
> 13
>
> 2. **一个字是否可以存放在任何字节边界上(对齐问题)**
>
> 假设对s个字节长的对象访问地址为A，如果A mod s =0 称为边界对齐
>
> 在许多体系结构中，字的起始地址必须是4的倍数，双字的起始地址必须是8的倍数。该要求称为对齐限制。

边界对齐的原因是存储器本身读写的要求，存储器本身读写通常就是边界对齐的，对于不是边界对齐的对象的访问可能要导致存储器的两次访问，然后再拼接出所需要的数。以32位机为例，它每次取32个位，也就是4个字节。以int型数据为例，如果它在内存中存放的位置按4字节对齐，也就是说1个int的数据全部落在计算机一次取数的区间内，那么只需要取一次就可以了。如图2-1。如果不对齐，很不巧，这个int数据刚好跨越了取数的边界，这样就需要取两次才能把这个int的数据全部取到，如图2-2，这样效率也就降低了。

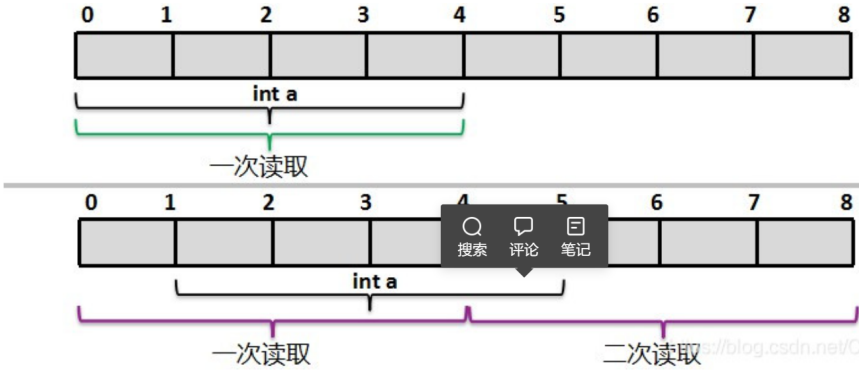


图2-1

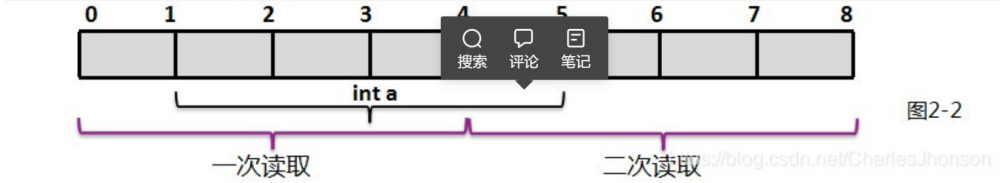


图2-2

未字节对齐也可能发生异常。某些平台对特定类型的数据只能从特定地址开始存取，而不允许其在内存中任意存放。例如，Motorola 68000 处理器不允许16位的字存放在奇地址，否则会触发异常，因此在这种架构下编程必须保证字节对齐。

- RISC-V和x86没有对齐要求，但是MIPS有对齐要求：即要求字的起始地址必须是4的倍数，而双字的起始地址必须是8的倍数。对齐限制会使数据传输更快。

寻址方式:

vs. 物理地址

Mode	Example	Meaning	When used
寄存器寻址 Register	Add R1, R2	R1 ← R1 + R2	Values in registers
立即数寻址 Immediate	Add R1, 100	R1 ← R1 + 100	For constants
寄存器间接寻址 Register Indirect	Add R1, (R2)	R1 ← R1 + Mem(R2)	R2 contains address
带偏移量的间接寻址 Displacement	Add R1, (R2+16)	R1 ← R1 + Mem(R2+16)	Address local variables
绝对寻址 Absolute	Add R1, (1000)	R1 ← R1 + Mem(1000)	Address static data
相对基址变址寻址 Indexed	Add R1, (R2+R3)	R1 ← R1 + Mem(R2+R3)	R2=base, R3=index
比例变址寻址 Scaled Index	Add R1, (R2+s*R3)	R1 ← R1 + Mem(R2 + s*R3)	s = scale factor = 2, 4, or 8
后增寄存器间接寻址 Post-increment	Add R1, (R2)+	R1 ← R1 + Mem(R2) R2 ← R2 + s	Stepping through array s = element size
前增寄存器间接寻址 Pre-decrement	Add R1, -(R2)	R2 ← R2 – s R1 ← R1 + Mem(R2)	Stepping through array s = element size

- 在现代计算机中操作以**一个字节（8bits）**为基本单位（不同的体系结构对字（word）的定义不同(RISC V中 1word = 4 bytes))
- 字地址之间有**4个字节**的距离

  字的地址为字内最低位字节的地址（小端模式）

  按字对齐，地址最后两位为0（4的倍数）

....	.....	....	....	....	....
15	高字节	....	....	低字节	1100
11	高字节	....	....	低字节	1000
7	高字节	....	....	低字节	0100
3	高字节	....	....	低字节	0000

- C语言会自动按照数据类型来计算地址，在汇编中需要程序员自己计算

- **数据传输指令**：
  - `lw  reg, offset(bAddrReg)` #从存储器**取"字(word)"**到寄存器
  - `sw  reg, offset(bAddrReg)` #从寄存器**存"字(word)"**到存储器

    注意：bAddrReg+offset 按照字进行对齐，即4的倍数
  - 双字：`ld/sd`
  - 半字：`lh/sh`
  - 字符：`lb/sb`

  例如整数的数字：每个整数32位 = 4字节

  示例：

例题 | **使用 load 和 store 编译生成指令**

假设变量 h 存放在寄存器 x21 中，数组 A 的基址存放在寄存器 x22 中。C 赋值语句的 RISC-V 汇编代码是什么？

```
A[12] = h + A[8];
```

答案 | 虽然在 C 语句中只有一个操作，但是现在在内存中有两个操作数，所以我们需要更多的 RISC-V 指令。前两个指令与前一个例子相同，只是这次我们在加载指令中使用了字节寻址正确的偏移量来选择 A[8]，并且加法指令将总和放在寄存器 x9 中：

```
ld    x9, 64(x22)    // Temporary reg x9 gets A[8]
add   x9, x21, x9    // Temporary reg x9 gets h + A[8]
```

最后的指令使用 96（8×12）作为偏移量，寄存器 x22 作为基址寄存器，将总和存储到 A[12] 中。

```
sd    x9, 96(x22)    // Stores h + A[8] back into A[12]
```

```
int A[100];                          (in C)
A[10] = A[3] + a;
```

（**RISC-V中一个字是32位，int类型是32位**）

```
# address of int A[] -> x3,   a -> x2        (in RISC-V)
    lw    x10, 12(x3)      # x10 = A[3]
    add   x10, x2, x10     # x10 = A[3]+a
    sw    x10, 40(x3)      # A[10] = A[3]+a
```

- **传输一个字节数据**：
  - 使用字类型指令，配合位的掩码（0xFF）来操作字节数据

    `lw x11, 0(x1)`

    `andi x11, x11, 0xFF` # lowest
  - 使用专门的字节传输指令

    `lb x11, 0(x1)`

    `sb x11, 0(x1)`
  - 字节传输指令无需字对齐

- **字节数据传输指令：**

  lb/sb使用的是**低字节**

  - 如果是sb指令，高56位(RV64)/24位(RV32)**被忽略（仅操作低位）**
  - 如果是lb指令，高56位(RV64)/24位(RV32)**做符号扩展**（同最右端符号相同）

    - 例如：mem[x1]中存储的数据为0x1020304050607080

    (RV64)　lb x11, **1**(x1)　#x11=0x0000000000000**70**

    lb x12, **0**(x1)　#x12=0xFFFFFFFFFFFFF**80**

    sb x12, **2**(x1)　#mem[x1]=0x1020304050**80**7080



- **半字数据传输指令**

  `lh reg, off(bAddr)`　#load half word

  `sh reg, off(bAddr)`　#store half word

  lhu、shu 无符号半字传输指令

  - off(bAddr)应该是2的倍数（对齐）
  - sh指令中高16位(RV32)/高48(RV64)忽略
  - lh指令中高16位(RV32)/高48(RV64)做符号扩展
  - lbu、lhu等带尾缀u的load类指令，**高位都做0扩展**

- **总结：**

  - 从地址为rs1的主存读字节/半字/字：符号扩展为64位存入rd
    - lb rd, offset(rs1)　#load byte　对于RV32则扩展为32位
    - lh rd, offset(rs1)　#load halfword
    - lw rd, offset(rs1)　#load word
  - 从主存读一个无符号的字节/半字/字：零扩展为64位存入rd
    - lb**u** rd, offset(rs1)
    - lh**u** rd, offset(rs1)
    - lw**u** rd, offset(rs1)
  - 将一个字节/半字/字写入主存：保存最右端的8/16/32位
    - sb rs2, offset(rs1)　# 保存最右端的8位　（低8位）
    - sh rs2, offset(rs1)　# 保存最右端的16位（低16位）
    - sw rs2, offset(rs1)　# 保存最右端的32位（低32位）



比较指令

`slt、slti、sltu、sltiu`

- **有符号数比较指令：**`slt,slti`

  - Set Less Than (slt：通过判断两个操作数的大小来对目标寄存器进行设置，**有符号比较**)

    `slt　dst,reg1,reg2`　#如果reg1中的值 < reg2中的值, dst = 1, 否则dst = 0

  - Set Less Than Immediate (slti)

    `slti dst,reg1,imm`　#如果reg1中的值<立即数imm, dst=1, 否则dst = 0

- **无符号数比较指令：**`sltu,sltiu`

`sltu dst,src1,src2`

`sltiu dst,src,imm`

- 示例：

```
addi   x10,x0,-1     #x10=0xFFFFFFFF
slti   x11,x10,1     #x11=0x00000001(-1< 1)
sltiu  x12,x10,1     #x12=0x00000000(2^32-1 > 1)
```

条件（跳转）分支

`beq、bne、blt、bge、bltu、bgeu`

条件分支指令：一条指令，先检测一个值，然后根据检测结果允许后续控制流转移到程序中的一个新地址。

不同于C语言中用{ 和 } 定义语句块开始和结束，RISC-V 汇编通过**标记（Label）**定义语句块起始。其中标记后面加一个冒号（例如 Loop: ），汇编的控制流就是跳转到标记的位置。

- 条件分支（跳转）指令：条件为真则转到标签所指的语句执行，否则顺序执行
  - `beq reg1,reg2,label` #branch if equal

    如果reg1中的值=reg2中的值, 程序跳转到label处继续执行
  - `bne reg1,reg2,label` #branch if not equal

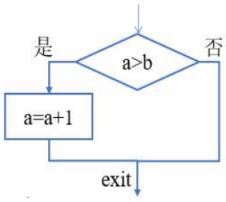
    如果reg1 ≠ reg2, 程序跳转到label处继续执行
  - `blt reg1,reg2,label` #branch if less than

    如果reg1 < reg2, 程序跳转到label处继续执行
  - `bge reg1,reg2,label` #branch if greater than or equal

    如果reg1 >= reg2, 程序跳转到label处继续执行
  - 无符号版本命令有：`bltu、bgeu`

示例：

```
if (a > b) a += 1;  //（C语言）
.....
```




```
假设：a保存在x22，b保存在x23          （RISC-V）
    bge x23, x22, Exit          # 当b >= a时跳转
    addi x22, x22, 1            # a=a+1
Exit: ....
```

可见，对于RSIC-V而言，要先说这个条件不符合（取反）然后跳转，中间空出来，再填你想要的语句。这和c语言相反。

示例：

例题 | 将 if-then-else 语句编译为条件分支指令

在下面的代码段中，f、g、h、i 和 j 是变量。如果五个变量 f 到 j 对应于 x19 到 x23 这 5 个寄存器，这个 C 语言的 if 语句编译后的 RISC-V 代码是什么？

```
if (i == j) f = g + h; else f = g - h;
```

答案 | 图 2-9 显示了 RISC-V 代码应该执行的操作的流程图。第一个表达式比较寄存器中两个变量是否相等。如果 i 和 j 相等则跳转，那么需要一条 beq 指令。一般来说，如果我们测试相反的条件来进行跳转，代码将更有效率，那么需要一条 bne 指令。代码如下：

```
bne x22, x23, Else   // go to Else if i ≠ j
```

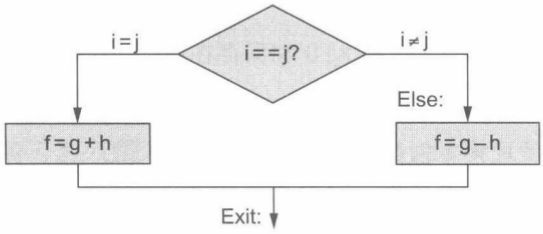


图 2-9　上述 if 语句的操作流程图。左边方框对应 if 语句的 then 部分，右边方框对应 else 部分

下一个赋值语句执行单个操作，并且如果所有操作数都分配给了寄存器，则它只是一条指令：

```
add x19, x20, x21       // f = g + h (skipped if i ≠ j)
```

现在看看 if 语句的结尾部分。本例引入了另一种分支，通常称为无条件分支。该指令表示在遇到该指令时，程序必须分支。在 RISC-V 中表达无条件分支的一种方法是使用条件始终为真的条件分支：

```
beq x0, x0, Exit       // if 0 == 0, go to Exit
```

if 语句的 else 部分中的赋值语句可以再次编译成单一指令。我们只需要在该指令中附加标签 Else。另外，在该指令之后还需 Exit 标签，表示 if-then-else 编译后的代码结束：

```
Else:sub x19, x20, x21    // f = g - h (skipped if i = j)
Exit:
```

示例：

例题 | 编译一个 C 语言的 while 循环

下面是一个 C 语言的常见循环：

```
while (save[i] == k)
        i += 1;
```

假设 i 和 k 对应于寄存器 x22 和 x24，数组的基址保存在 x25 中。与此 C 语言代码相对应的 RISC-V 汇编代码是什么？

答案 | 第一步是将 save[i] 加载到临时寄存器中。在将 save[i] 加载到临时寄存器之前，需要得到它的地址。在将 i 加到数组 save 的基址以形成地址之前，由于字节寻址问题，必须将索引 i 乘以 8。幸运的是，我们可以使用左移，因为左移 3 位相当于乘以 2^3 或 8（参见上一节）。我们需要给指令添加标签 Loop，以便我们可以在循环结尾跳转回该指令：

```
Loop: slli x10, x22, 3      // Temp reg x10 = i * 8
```

要获得 save[i] 的地址，我们需要将 x10 和保存在 x25 中的基址相加：

```
add x10, x10, x25       // x10 = address of save[i]
```

现在我们可以利用这个地址将 save[i] 加载到临时寄存器了：

```
ld x9, 0(x10)       // Temp reg x9 = save[i]
```

下一条指令将进行循环判断，如果 save[i] ≠ k 则退出：

```
bne x9, x24, Exit     // go to Exit if save[i] ≠ k
```

接下来的指令将 i 加 1：

```
addi x22, x22, 1       // i = i + 1
```

循环的结尾回到循环顶部的 while 判断。我们只需在它后面添加 Exit 标签：

```
beq x0, x0, Loop       // go to Loop
Exit:
```

C代码：

```
long long int save[100];
.....
while (save[i] == k) i += 1;
...
```

假设：i存储在x22中，k存储在x24中，save数组元素的地址保存在x25中。



RISC-V64I 代码：

```
Loop: slli x10, x22, 3     #8*i
      add x10, x10, x25    #save[]+8*i
      ld x9, 0(x10)        #save[i]—>x9
      bne x9, x24, Exit    #save[i]!=k Exit
      addi x22, x22, 1     #i=i+1
      beq x0, x0, Loop     # 循环
Exit: …
```

示例：用RV64汇编实现C语言for循环结构

C语言

```
long long int A[20];
long long int sum = 0;
for (long long int i = 0; i < 20; i++)
        sum += A[i];
//假设数组A的首地址保存在 x8，
//sum保存在x10中
```



```
    add x9, x8, x0     # x9=&A[0]
    add x10, x0, x0    # sum=0
    add x11, x0, x0    # i=0
    addi x13, x0, 20   # x13=20
Loop:
    bge x11, x13, Done
    ld x12, 0(x9)      # x12=A[i]
    add x10, x10, x12
    addi x9, x9, 8     # &A[i+1]
    addi x11, x11, 1   # i++
    beq x0, x0, Loop
Done:
```

无条件跳转

`jal、jalr`

> **程序计数器（PC）**：用来保存当前执行指令地址的寄存器，也叫指令地址寄存器
>
> **跳转-链接指令**：跳转到某个地址的同时将下一条指令的地址保存到目的寄存器rd

- `jal rd, offset` #（jump and link）

  将下一条指令的地址（即PC+4）保存在寄存器rd(一般用x1/ra)

  跳转到目标地址(PC+立即数偏移量，有规定的转换规则)

  可用于**过程调用**

- `jalr rd, offset(rs1)` #(jump and link register)

  把PC+4存到rd中

  类似jal，但跳转到rs1+offset地址处的指令（**更远**）

  可以用于**过程返回**

- 如果rd用x0，即`jal x0 label`，相当于只跳转不返回（丢弃返回地址）

RISC-V 中的有符号与无符号

有符号和无符号在3个上下文环境中使用

–Signed vs. unsigned bit extension 符号扩展

- `lb, lh` vs. `lbu, lhu`

–Signed vs. unsigned comparison 比较

- `slt, slti` vs. `sltu, sltiu`

–Signed vs. unsigned branch 比较

- `blt, bge` vs. `bltu, bgeu`

伪指令

[汇编语言程序设计第二篇——80X86汇编语言伪指令 - 知乎 (zhihu.com)](https://zhuanlan.zhihu.com/p/34102124)

- 伪指令通过汇编语言的变化或者组合来实现，没有对应机器码的指令，最终不被CPU所执行，而是由**编译器**执行的指令。编译器根据伪指令来进行相关的编译工作。

  例如：`mv dst, src`（被翻译为addi dst, src, 0 或者 add dst, src）

  load immediate (li)：`li dst, imm` 装入一个立即数到 dst寄存器（被翻译为: addi dst,x0,imm）

  load address (la)：`la dst, label` 装入由Label指定的地址到 dst

  伪指令（不是硬件实现）与硬件指令的区别?

  硬件指令中所有指令都在硬件中直接实现，可以直接执行；而伪指令是为了方便汇编语言程序员使用，每一条伪指令会被翻译为**1条或者多条**硬件指令。

Pseudo	Real
`nop`	`addi x0, x0, 0`
`not rd, rs`	`xori rd, rs, -1`
`beqz rs, offset`	`beq rs, x0, offset`
`bgt rs, rt, offset`	`blt rt, rs, offset`
`j offset`	`jal x0, offset`
`ret`	`jalr x0, x1, offset`
`call offset` (if too big for just a `jal`)	`auipc x6, offset[31:12]` `jalr x1, x6, offset[11:0]`
`tail offset` (if too far for a `j`)	`auipc x6, offset[31:12]` `jalr x0, x6, offset[11:0]`

示例：

- 伪指令+条件分支转移指令

C语言

```
if (i == j)
    f = g + h;
else
    f = g – h;
```

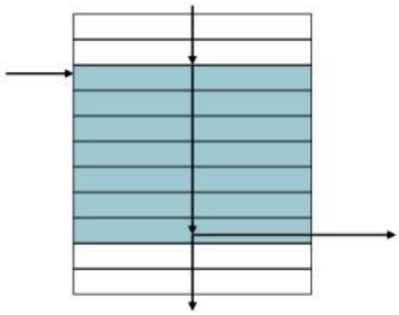
RISC V

f→x10, g→x11, h →x12, i →x13, j →x14

```
    bne x13, x14, Else
    add x10, x11, x12
    j Exit   #伪指令
Else: sub x10, x11, x12
Exit:
```

基本块

- 基本块是这样的指令序列：没有嵌入分支（除非在末尾），没有分支目标（除非在开头）。类似于C语言中的顺序执行语句



为什么需要基本块？ 编译器可以识别基本块以进行优化；先进的处理器能够加速基本块的执行。

- 例：字符串复制

例题 | 编译字符串复制程序，展示如何使用 C 语言字符串

strcpy 过程将字符串 y 复制到字符串 x，C 语言使用 null 字节标记字符串结束：

```
void strcpy (char x[], char y[])
{
   size_t i;
   i = 0;
   while ((x[i] = y[i]) != '\0') /* copy & test byte */
      i += 1;
}
```

编译后的 RISC-V 汇编代码是什么？

答案 | 下面是基本的 RISC-V 汇编代码段。假设数组 x 和 y 的基址存放在 x10 和 x11 中，而 i 在 x19 中。strcpy 调整栈指针，然后将保存的寄存器 x19 保存在栈中：

```
strcpy:
    addi  sp, sp, -8       // adjust stack for 1 more item
    sd    x19, 0(sp)       // save x19
```

将 i 初始化为 0，下一条指令通过 0 加 0 将 x19 设为 0 并将结果放在 x19 中：

```
add  x19, x0, x0     // i = 0+0
```

这是循环的开始。y [i] 的地址首先通过将 i 加到 y [] 来形成：

```
L1: add  x5, x19, x11  // address of y[i] in x5
```

注意，我们不必将 i 乘以 8，因为 y 是字节数组而不是双字，如前面的例子中那样。

为了加载 y[i] 中的字符，我们使用无符号加载字节，将字符放入 x6 中：

```
lbu  x6, 0(x5)  // x6 = y[i]
```

类似的地址计算将 x [i] 的地址放在 x7 中，然后 x6 中的字符存储在那个地址中。

```
add    x7, x19, x10     // address of x[i] in x7
sb     x6, 0(x7)        // x[i] = y[i]
```

接下来，如果字符为 0，则退出循环。也就是说，如果它是字符串的最后一个字符，我们退出：

```
beq x6, x0, L2
```

如果不是，递增 i 继续循环：

```
addi  x19, x19, 1   // i = i + 1
jal   x0, L1        // go to L1
```

如果不继续循环，它就是字符串的最后一个字符；我们恢复 x19 和栈指针，然后返回。

```
L2: ld      x19, 0(sp)     // restore old x19
    addi    sp, sp, 8      // pop 1 doubleword off stack
    jalr    x0, 0(x1)      // return
```

```
    strcpy:addi sp,sp,-8        # 调整栈，留出1个双字的空间
    sd x19,0(sp)                # x19入栈
    add x19,x0,x0               # i=0  #i存在x19中
func1: add x5,x19,x11           # x11为数组src首地址，x5 = src[i]的地址
    lbu x6,0(x5)                # x6最低字节为src[i]
    add x7,x19,x10              #x10保存dest首地址，  x7 = dest[i]的地址
    sb x6,0(x7)                 # dest[i] = src[i]
    beq x6,x0,func2             # 若src[i] == '\0'，则退出
    addi x19,x19,1              # i = i + 1
    jal x0,func1                # 下一次循环
func2: ld x19,0(sp)             # 恢复x19原值
    addi sp,sp,8                # 从栈中弹出1个双字
    jalr x0,0(x1)               # 返回  #x1（ra）
```



过程调用

- 过程（子程序）调用的步骤：（调用者caller vs. 被调用者callee）
  - (Caller)将参数放在过程可以访问的寄存器x10~x17/a0-a7
  - (Caller)将控制权转交给过程(Callee)
  - (Callee)获取过程所需的存储资源(包括保存现场)
  - (Callee)执行过程中的操作
  - (Callee)将结果值放在调用程序可以访问到的寄存器s0-s11
  - (Callee)返回调用位置(x1/ra中的地址)，控制权交还调用者(Caller)
- 过程调用指令：
  - **过程调用**：`jal x1, ProcedureAddress`(跳转和链接)

    将**下一条指令的地址**保存在x1/ra中

    跳转到目标地址ProcedureAddress
  - **过程返回**：`jalr x0, 0(x1)`(寄存器跳转和链接)

    类似jal，但跳转到**0 + x1**中保存的地址

    把x0用作目的寄存器（实际x0不会被改变），不会再返回了（可以理解成c语言中的return）

    同样可用于跳转到计算出的位置
- 过程调用相关的寄存器
  - 返回地址寄存器 x1(ra)，用于返回到起始点
  - 参数寄存器 x10-x17(a0-a7)，用于传递参数或者返回值

    返回值寄存器 x10-x11(a0-a1)
  - 保存寄存器 x8,x9,x18-x27(s0,s1,s2-s11)，在调用过程中必须被保存
  - 临时寄存器 x5-x7,x28-x31(t0-t2,t3-t6)
  - **栈指针 x2(sp)**

保存	不保存
保存寄存器：x8～x9，x18～x27	临时寄存器：x5～x7，x28～x31
栈指针寄存器：x2(sp)	参数/结果寄存器：x10～x17
帧指针：x8(fp)	
返回地址：x1(ra)	
栈指针以上的栈	栈指针以下的栈

RISC-V过程调用（人为设定返回地址）：

C语言函数调用

```
int function(int a ,int b)        # a, b: s0, s1  调用者把参数存于s0,s1
{  return (a+b); }
```

```
1000 mv a0, s0
1004 mv a1, s1
1008 addi ra, zero, 1016  # ra = 1016
1012 j sum   # jump to sum
1016 …                #跳转返回的下条指令
…    …
2000 sum: add a0, a0, a1
2004 jr ra    # jalr x0,ra,0
jr: jump register    #PC=rs1
```

- 返回为什么用**jr**而不用j？
  - 伪指令j跳转到给定的地址
  - sum可能在多处被调用，不能返回到一个固定地址。
  - 调用sum过程必须能够返回调用语句的下一句。

栈的使用

> 由于在任务完成后必须掩盖踪迹，调用者所需的所有寄存器都必须恢复到调用该过程之前所存储的值。这种情况是需要将寄存器换出到存储器中的一个例子。换出寄存器的理想数据结构是**栈(stack)**——一种后进先出的队列。栈需要一个指向栈中最新分配地址的指针，以指示下一个过程应该放置换出寄存器的位置或寄存器旧值的存放位置。在RISC-V中，栈指针(stack pointer）是**寄存器x2**，也称为**sp**，用来保存栈顶地址（sp的初值不是硬件决定的，是软件约定）。栈指针按照每个被保存或恢复的寄存器按双字进行调整。栈应用非常广泛，因而传送数据到栈或从栈传输数据都具有专业术语：将数据放入栈中称为**压栈**，从栈中移除数据称为**弹栈**。
>
> 按历史惯例，RISC-V中栈按照从高到低的地址顺序增长 （即**栈底位于高地址，栈顶位于低地址**）
>
> - 压栈（入栈）向下移动栈指针（减 sp）
> - 弹栈（出栈）向上移动栈指针（增 sp）

- RISC V无专用的入栈和出栈指令
- **如何使用栈?**
  - 寄存器sp总是指向栈中最后使用的空间
  - 首先将这个指针减少所需的空间量，然后用需要保存信息填充它

    例如，保存寄存器x9的内容到栈中(RV-32)：

    `addi sp, sp, -4` # 将栈指针sp向低地址移动4个字节

    `sw x9, 0(sp)` # 将寄存器x9中的内容写入栈

例如：

例题 | 编译一个没有调用其他过程的 C 过程

将 2.2 节的例题转化为一个 C 过程：

```
long long int leaf_example (long long int g, long long
int h, long long int i, long long int j)
{
      long long int f;

       f = (g + h) - (i + j);
       return f;
}
```

编译后的 RISC-V 汇编代码是什么呢？

答案 | 参数变量 g、h、i 和 j 对应于参数寄存器 x10、x11、x12 和 x13，f 对应于 x20。编译后的程序从如下过程标号开始：

```
leaf_example:
```

下一步是保存该过程使用的寄存器。过程体中的 C 赋值语句与之前的例题相同，使用两个临时寄存器（x5 和 x6）。因此，需要保存三个寄存器：x5、x6 和 x20。通过在栈中创建三个双字（24 字节）空间并将数据存入，实现将旧值“压”入栈中：

```
addi sp, sp, -24          // adjust stack to make room for 3 items
sd   x5, 16(sp)           // save register x5 for use afterwards
sd   x6, 8(sp)            // save register x6 for use afterwards
sd   x20, 0(sp)           // save register x20 for use afterwards
```

图 2-10 展示了过程调用之前、之中和之后栈的情况。

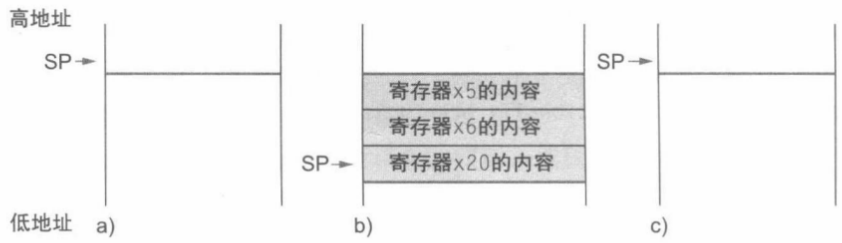


图 2-10　过程调用之前（a）、之中（b）和之后（c）栈指针以及栈的值。栈指针总是指向栈“顶”，或者图中栈的最后一个双字

```
add x5, x10, x11    // register x5 contains g + h
add x6, x12, x13    // register x6 contains i + j
sub x20, x5, x6     // f = x5 - x6, which is (g + h) - (i + j)
```

为了返回 f 的值，将其复制到一个参数寄存器中：

```
addi x10, x20, 0 // returns f (x10 = x20 + 0)
```

在返回之前，通过从栈中“弹出”数据来恢复寄存器的三个旧值：

```
ld x20, 0(sp)     // restore register x20 for caller
ld x6, 8(sp)      // restore register x6 for caller
ld x5, 16(sp)     // restore register x5 for caller
addi sp, sp, 24   // adjust stack to delete 3 items
```

通过一个使用返回地址的跳转寄存器结束过程：

```
jalr x0, 0(x1)       // branch back to calling routine
```

先前示例使用了临时寄存器，并假设其旧值必须被保存和恢复。为了避免保存和恢复一个其值从未被使用过的寄存器（通常为临时寄存器），RISC-V 软件将 19 个寄存器分成两组：

- x5 ~ x7 以及 x28 ~ x31：临时寄存器，在过程调用中不被被调用者（被调用的过程）保存。
- x8 ~ x9 以及 x18 ~ x27：保存寄存器（saved register），在过程调用中必须被保存。（一旦使用，由被调用者保存并恢复）

这一简单约定减少了寄存器换出。在上述例子中，由于调用者不希望在过程调用中保存寄存器 x5 和 x6，可以从代码中去掉两次存储和两次载入。但仍须保存并恢复 x20，因为被调用者必须假设调用者需要该值。

- **RISC-V中调用过程时寄存器的使用：**

  当被调用过程从执行中返回时，调用过程需要知道哪些寄存器可能已经更改，哪些寄存器保证不变。RISC-V过程调用时对需要保护现场的寄存器有如下约定：

  - 保存：`x1(ra), sp, gp, tp, x8-x9(s0-s1),x18-x27(s2-s11)`
  - 不保存：`x10-x17(a0-a7), x5-x7(t0-t2), x28-x31(t3-t6)`

- **嵌套过程**：

  例如：

```c
int numSquare (int x, int y){
    return mult (x, x)+y;
}
```

```
• numSquare:        #a0=x，a1=y
      addi sp, sp, -8   # 预留栈空间
入栈  sw ra, 4(sp)      # ra寄存器入栈
      sw al, 0(sp)      # 参数寄存器a1值y入栈
      mv al, a0         # a1=a0=x，为mult(x, x)准备参数
      jal mult          # 调用mult计算x*x
      lw al, 0(sp)      # 出栈，恢复a1的值y
      add a0, a0, a1    # x*x+y
出栈  lw ra, 4(sp)      # 返回地址出栈
      addi sp, sp, 8    # 恢复栈指针
      jr ra
  mult:…
```

  例如：

```c
long long int fact (long long int n){
    if (n < 1)      return 1;
    else    return n * fact(n - 1);
}
```

答案 | 参数变量 n 对应参数寄存器 x10。编译后的程序从过程的标签开始，然后在栈中保存两个寄存器，返回地址和 x10：

```
fact:
    addi sp, sp, -16   // adjust stack for 2 items
      sd x1, 8(sp)     // save the return address
      sd x10, 0(sp)    // save the argument n
```

第一次调用 fact 时，sd 保存程序中调用 fact 的地址。下面两条指令测试 n 是否小于 1，如果 n ≥ 1 则跳转到 L1。

```
addi    x5, x10, -1        // x5 = n - 1
bge     x5, x0, L1         // if (n - 1) >= 0, go to L1
```

如果 n 小于 1，fact 将 1 放入一个值寄存器中以返回 1：它将 1 加 0 并将和存入 x10 中。然后从栈中弹出两个已保存的值并跳转到返回地址：

```
addi    x10, x0, 1    // return 1
addi    sp, sp, 16    // pop 2 items off stack
jalr    x0, 0(x1)     // return to caller
```

在从栈中弹出两项之前，可以加载 x1 和 x10。因为当 n 小于 1 时 x1 和 x10 不会改变，所以跳过这些指令。

如果 n 不小于 1，则参数 n 递减，然后用递减后的值再次调用 fact：

```
L1: addi x10, x10, -1  // n >= 1: argument gets (n – 1)
    jal  x1, fact      // call fact with (n – 1)
```

下一条指令是 fact 的返回位置，其结果在 x10 中。现在旧的返回地址和旧的参数与栈指针一起被恢复：

```
addi  x6, x10, 0    // return from jal: move result of fact
                       (n - 1) to x6:
ld    x10, 0(sp)    // restore argument n
ld    x1, 8(sp)     // restore the return address
addi  sp, sp, 16    // adjust stack pointer to pop 2 items
```

接下来，参数寄存器 x10 得到旧参数与 fact(n-1) 结果的乘积，目前在 x6 中。假设有一个乘法指令可用，尽管在第 3 章才会涉及：

```
mul     x10, x10, x6     // return n * fact (n – 1)
```

最后，fact 再次跳转到返回地址：

```
jalr     x0, 0(x1)     // return to the caller
```

(其余内容详见PPT)

RISC-V指令表示

RISC-V六种指令格式

- 指令格式：用二进制代码表示指令的结构形式（要求计算机处理什么数据？要求计算机对数据做什么处理？计算机怎样才能得到要处理的数据？）

  指令字分成多个“字段”（一部分位），每个字段有特定的含义和作用

操作码字段	操作数字段

  - 操作码字段长度决定指令系统规模（指令总数）

    每条指令对应一个操作码（操作码越短，则操作数越长，表示的范围越大）

    定长操作码：$Length_{OP} = \log_2 n$（RSIC-V）

变长操作码：操作码向不用的地址码字段扩展（X86）

- 操作数字段可能有多个：

  寻址方式字段：长度与方式总数有关，可能隐含在操作码字段

  地址码字段：作用及影响，长度与寻址方式有关

- RISC-V指令处理的大多数数据都是**字**
- 指令字长、机器字长、存储字长 (二进制位数)
  - **指令字长**：指令中二进制代码的总位数
  - **机器字长**：直接处理的二进制位数，一般等于寄存器总位数
  - **存储字长**：一个存储单元存储的二进制代码的长度
- RISC-V的指令都是**32位**（无论是RV32、RV64还是RV128）

	int[31:25]	int[24:20]	int[19:15]	int[14:12]	int[11:7]	inst[6:0]
R型	func7	rs2编号	rs1编号	func3	rd编号	opcode
I型	imm[11:5]	imm[4:0]	rs1	func3	rd	opcode
S型	imm[11:5]	rs2	rs1	func3	imm[4:0]	opcode
B型	imm[12,10:5]	rs2	rs1	func3	imm[4:1,11]	opcode
J型	imm[20,10:5]	imm[4:1,11]	imm[19:12]		rd	opcode
U型	imm[31:12]				rd	opcode

inst[4:2] inst[6:5]	000	001	010	011	100	101	110	111 (> 32b)
00	LOAD	LOAD-FP	*custom-0*	MISC-MEM	OP-IMM	AUIPC	OP-IMM-32	48b
01	STORE	STORE-FP	*custom-1*	AMO	OP	LUI	OP-32	64b
10	MADD	MSUB	NMSUB	NMADD	OP-FP	*reserved*	*custom-2/rv128*	48b
11	BRANCH	JALR	*reserved*	JAL	SYSTEM	*reserved*	*custom-3/rv128*	≥ 80b

Table 24.1: RISC-V base opcode map, inst[1:0]=11

R型指令

用于**寄存器 - 寄存器**操作

funct7	rs2	rs1	funct3	rd	opcode
7 位	5 位	5 位	3 位	5 位	7 位

32位指令字，分为6个不同位数的字段。以下是 RISC-V指令中每个字段名称的含义：

- opcode(操作码)：指令的基本操作，这个缩写是它的惯用名称，表示是哪类指令。
- rd：目的操作数寄存器，用来存放操作结果。
- funct3：一个另外的操作码字段。

  funct7 ： 一个另外的操作码字段。

  funct7+funct3：这两个字段结合操作码描述要执行的操作
- rs1：第一个源操作数寄存器。
- rs2：第二个源操作数寄存器

  rs1、rs2、rd每个字段保存一个5位无符号整数（0-31，对应于寄存器 编号0-31）

0000000	rs2	rs1	000	rd	0110011	add
0100000	rs2	rs1	000	rd	0110011	sub
0000000	rs2	rs1	001	rd	0110011	sll
0000000	rs2	rs1	010	rd	0110011	slt
0000000	rs2	rs1	011	rd	0110011	sltu
0000000	rs2	rs1	100	rd	0110011	xor
0000000	rs2	rs1	101	rd	0110011	srl
0100000	rs2	rs1	101	rd	0110011	sra
0000000	rs2	rs1	110	rd	0110011	or
0000000	rs2	rs1	111	rd	0110011	and

示例：`add x18, x19, x10`：0000000 01010 10011 000 10010 011001

I型指令

用于**短立即数**和**访问内存 load** 操作

immediate	rs1	funct3	rd	opcode
12 位	5 位	3 位	5 位	7 位

立即数指令：

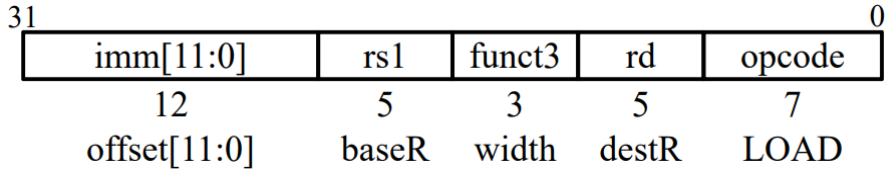
- I型指令的rs1、funct3、rd、opcode字段与R型相同
- rs2和funct7被12位有符号立即数imm[11:0]替换
  - 12位immediate字段为**补码值**，所以它可以表示从-2^{11}到2^{11}-1之间的整数
- 在算术运算前，立即数总是进行**符号扩展**（正数补0，负数补1）

imm[11:0]		rs1	000	rd	0010011	addi
imm[11:0]		rs1	010	rd	0010011	slti
imm[11:0]		rs1	011	rd	0010011	sltiu
imm[11:0]		rs1	100	rd	0010011	xori
imm[11:0]		rs1	110	rd	0010011	ori
imm[11:0]		rs1	111	rd	0010011	andi
000000	shamt	rs1	001	rd	0010011	slli
000000	shamt	rs1	101	rd	0010011	srli
010000	shamt	rs1	101	rd	0010011	srai

> 一个高立即数位用于区分逻辑右移(SRLI)和算术右移(SRAI)
>
> “按立即数移位”仅将立即数低6位shamt用作移位量(最多移位63次)

示例：`addi x15, x1, -50`：111111001110 00001 000 01111 0010

Load类指令：



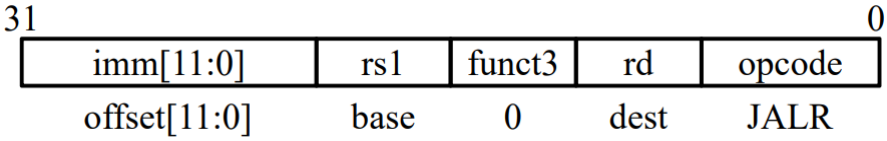
- immediate字段表示一个字节偏移量。12位有符号立即数加寄存器rs1中的基址，形成内存地址
- 从内存读取到的值存储在寄存器rd中
- width表示装载的数据位数宽度和是否考虑符号

imm[11:0]	rs1	000	rd	0000011	lb
imm[11:0]	rs1	001	rd	0000011	lh
imm[11:0]	rs1	010	rd	0000011	lw
imm[11:0]	rs1	011	rd	0000011	ld
imm[11:0]	rs1	100	rd	0000011	lbu
imm[11:0]	rs1	101	rd	0000011	lhu
imm[11:0]	rs1	110	rd	0000011	lwu

> lbu是“读取无符号字节”
>
> lh是“读取半字”，符号扩展以填充32/64位寄存器
>
> lhu是“读取无符号半字”，零扩展以填充32/64位寄存器

示例：`lw x14, 8(x2)`：000000001000 00010 010 01110 00000

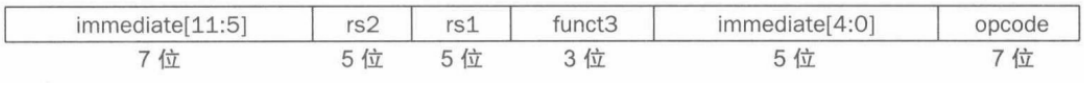
> **jalr指令**



- jalr rd, offset(rs1) #(jump and link register)
- 将PC + 4保存在rd中
- 设置 PC = rs1中的值 + offset
- 与load指令得到地址的方式类似

S型指令

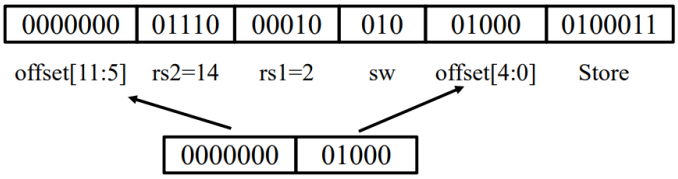
> 用于**访存 store** 操作



- 对应指令：`memop reg, offset(bAddrReg)`
- 存储指令需要读取两个寄存器，rs1中为内存地址（基址），rs2中为要写入的数据；offset为立即数形式的地址偏移量
- S型指令不会将值写入寄存器，所以没有rd
- RISC-V的设计决定是将低位的5位立即数移到其他指令中 的rd字段所在的位置——目的是保持rs1/rs2字段在同一位置

imm[11:5]	rs2	rs1	000	imm[4:0]	0100011	sb
imm[11:5]	rs2	rs1	001	imm[4:0]	0100011	sh
imm[11:5]	rs2	rs1	010	imm[4:0]	0100011	sw
imm[11:5]	rs2	rs1	011	imm[4:0]	0100011	sd

示例：`sw x14, 8(x2)`



三种基本指令总结：

R型指令	funct7	rs2	rs1	funct3	rd	opcode	示例
add (add)	0000000	00011	00010	000	00001	0110011	add x1, x2, x3
sub (sub)	0100000	00011	00010	000	00001	0110011	sub x1, x2, x3
I型指令	**immediate**		**rs1**	**funct3**	**rd**	**opcode**	**示例**
addi (add immediate)	001111101000		00010	000	00001	0010011	addi x1, x2, 1000
ld (load doubleword)	001111101000		00010	011	00001	0000011	ld x1, 1000 (x2)
S型指令	**immed-iate**	**rs2**	**rs1**	**funct3**	**immed-iate**	**opcode**	**示例**
sd (store doubleword)	0011111	00001	00010	011	01000	0100011	sd x1, 1000(x2)

图 2-6　2.5 节介绍的 RISC-V 体系结构。目前为止介绍的三种 RISC-V 指令格式是 R 型、I 型和 S 型。R 型格式有两个源寄存器操作数和一个目标寄存器操作数。I 型格式用一个 12 位的 immediate 字段替换了一个源寄存器操作数。S 型格式有两个源操作数和一个 12 位的 immediate 字段，但没有目标寄存器操作数。S 型的 immediate 字段分为两部分，最左边的字段是 11 ～ 5 位，最右边的字段是 4 ～ 0 位

综合例题：

例题 | 将 RISC-V 汇编语言翻译为机器语言

现在我们可以举一个例子来描述从程序员编写程序到计算机执行指令的整个过程。假设数组 A 的基址存放于 x10，h 存放于 x21，则赋值语句：

```
A[30] = h + A[30] + 1;
```

被编译成：

```
ld   x9, 240(x10)    // Temporary reg x9 gets A[30]
add  x9, x21, x9     // Temporary reg x9 gets h+A[30]
addi x9, x9, 1       // Temporary reg x9 gets h+A[30]+1
sd   x9, 240(x10)    // Stores h+A[30]+1 back into A[30]
```

这三条指令的 RISC-V 机器语言代码是什么？

答案 | 为了方便起见，我们首先使用十进制数来表示机器语言指令。从图 2-5 中，我们可以确定三条机器语言指令：

immediate	rs1	funct3	rd	opcode
240	10	3	9	3

funct7	rs2	rs1	funct3	rd	opcode
0	9	21	0	9	51

immediate	rs1	funct3	rd	opcode
1	9	0	9	19

immediate[11:5]	rs2	rs1	funct3	immediate[4:0]	opcode
7	9	10	3	16	35

ld 指令由操作码字段中的值 3（见图 2-5）和 funct3 字段中的值 3 共同标识。基址寄存器 10 在 rs1 字段中指定，目标寄存器 9 在 rd 字段中指定。用于选定 A[30]（240 = 30 × 8）的偏移量存放在 immediate 字段中。

接下来的 add 指令由操作码字段中的值 51、funct3 字段中的值 0 和 funct7 字段中的值 0 共同指定。三个寄存器操作数（9、21 和 9）存放于 rd、rs1 和 rs2 字段中。

随后的 addi 指令由操作码字段中的值 19 和 funct3 字段中的值 0 共同决定。寄存器操作数（9 和 9）存放在 rd 和 rs1 字段中，常量加数 1 存放于 immediate 字段中。

sd 指令由操作码字段中的值 35 和 funct3 字段中的值 3 共同标识。寄存器操作数（9 和 10）分别存放于 rs2 和 rs1 字段中。地址偏移量 240 分开存放于两个 immediate 字段之中。由于高位的 immediate 字段已经包含了低 5 位的值，所以可以通过除以 2^5 来分解偏移量 240。则高位的 immediate 字段存储除以 2^5 的商 7，低位的 immediate 字段存储余数 16。

因为 $240_{10} = 0000\ 1111\ 0000_2$，所以与上述十进制对应的二进制机器指令如下所示：

immediate	rs1	funct3	rd	opcode
000011110000	01010	011	01001	0000011

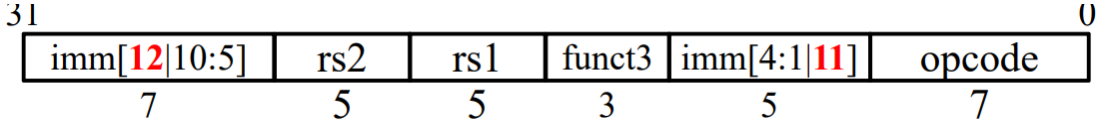
funct7	rs2	rs1	funct3	rd	opcode
0000000	01001	10101	000	01001	0110011

immediate	rs1	funct3	rd	opcode
000000000001	01001	000	01001	0010011

immediate[11:5]	rs2	rs1	funct3	immediate[4:0]	opcode
0000111	01001	01010	011	10000	0100011

B型指令

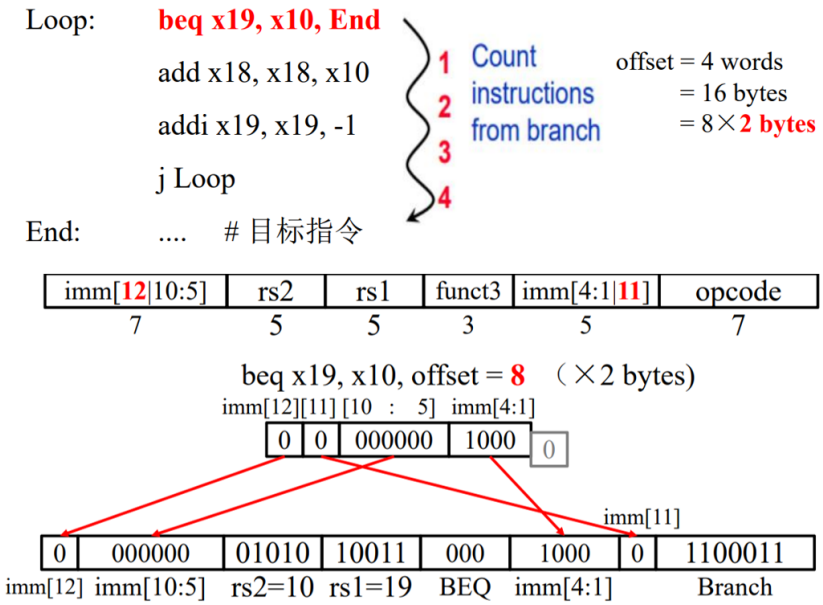
> 用于**条件跳转/分支/转移**操作`beq, bne, blt, bge, bltu, bgeu`。B型指令读取两个寄存器，但不写回寄存器（类似于S型指令）。最大分支转移距离与代码量有关



- 立即数字段（12位）作为相对于PC的**补码**偏移量
  - 若偏移量为以字节为单位，范围约为±2^{11}字节
  - 若偏移量为以字（指令）为单位，范围约为±2^{11}×4字节
- 折中考虑，B型指令的偏移量以**半字**为单位（**2字节为增量**）
  - 偏移量范围约为±2^{11}×2字节（-4096—4094字节）

imm[12|10:5]	rs2	rs1	000	imm[4:1|11]	1100011	BEQ
imm[12|10:5]	rs2	rs1	001	imm[4:1|11]	1100011	BNE
imm[12|10:5]	rs2	rs1	100	imm[4:1|11]	1100011	BLT
imm[12|10:5]	rs2	rs1	101	imm[4:1|11]	1100011	BGE
imm[12|10:5]	rs2	rs1	110	imm[4:1|11]	1100011	BLTU
imm[12|10:5]	rs2	rs1	111	imm[4:1|11]	1100011	BGEU

示例:



示例:

例题 | 描述机器语言中的分支偏移

2.7.1 节的 while 循环已编译为下列 RISC-V 汇编代码:

```
Loop:slli x10, x22, 3     // Temp reg x10 = i * 8
     add  x10, x10, x25   // x10 = address of save[i]
     ld   x9, 0(x10)      // Temp reg x9 = save[i]
     bne  x9, x24, Exit   // go to Exit if save[i] != k
     addi x22, x22, 1     // i = i + 1
     beq  x0, x0, Loop    // go to Loop
Exit:
```

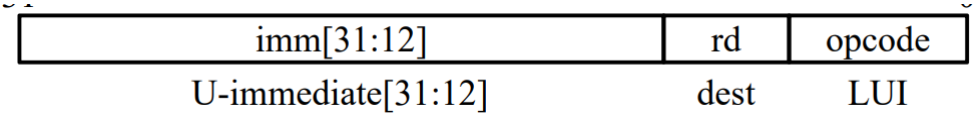
如果我们假设将循环的开始放在内存的 80000 处中，那么这个循环的 RISC-V 机器代码是什么?

答案 | 汇编指令及其地址如下:

地址	指令					
80000	0000000	00011	10110	001	01010	0010011
80004	0000000	11001	01010	000	01010	0110011
80008	0000000	00000	01010	011	01001	0000011
80012	0000000	11000	01001	001	01100	1100011
80016	0000000	00001	10110	000	10110	0010011
80020	1111111	00000	00000	000	01101	1100011

U型指令

> 用于**长立即数**操作



- 高20位为长度为20位的立即数字段（低12位通过addi来操作）
- rd：5位目的地寄存器字段
- **LUI(Load Upper Immediate)**：将长立即数写入目的寄存器

  AUIPC(Add Upper Immediate to PC)：将PC与长立即数，相加结果写入目的寄存器

LUI（取立即数高位）：

- LUI将长立即数写入目的寄存器的高20位，并清除低12位（将20位常数加载到寄存器的第31位到第12位。将第31位的值复制填充到最左边32位，最右边的12位用0填充）
- LUI与ADDI一起可在寄存器中创建任何32位值，例如：
  - `LUI t0, 0x87654` #t0 = 0x87654000
  - `ADDI t0, t0, 0x321` # 本条语句执行后t0 = 0x87654321
- 应用：

  • 如何创建 0xDEADBEEF?

  ```
  LUI    t2, 0xDEADB        # t2 = 0xDEADB000
  #ADDI  t2, t2, 0xEEF      #会出错？too large to fit in ...
  ADDI   t2, t2, 0xFFFFFEEF # t2 = 0xDEADAEEF?
  ```

  #ADDI 立即数总是进行**符号扩展**，如果高位为1，将从高位的20位中减去1

  • 伪指令 li x10, 0xDEADBEEF # 创建两条指令

  ```
  LUI    t1, 0xDEADC        # t1 = 0xDEADC000
  ADDI   t1, t1, 0xFFFFFEEF # t1 = 0xDEADBEEF
  ```

AUIPC：

- 将长立即数值加到PC并写入目的寄存器
- 用于PC相对寻址`Label: AUIPC rd, imm` #将Label地址+立即数imm存rd

示例：

写出产生64位常量0x1122334455667B88的RISC-V汇编代码，并将该值存储到寄存器x10(RISC V 64)。

```
lui x10, 0x11223          #x10=0x0000 0000 1122 3000
addi x10, x10, 0x344      #x10=0x0000 0000 1122 3344
slli x10, x10, 32         #x10=0x1122 3344 0000 0000
lui x5, 0x55668           #x5 = 0x0000 0000 5566 8000
addi x5, x5, 0xFFFFFB88   #x5 = 0x0000 0000 5566 7B88
add x10, x10, x5          #x10=0x1122 3344 5566 7B88
```

J型指令

> 用于**无条件跳转**操作
>
> `jal rd, offset` # 将PC+4写入目的寄存器rd中
>
> - j 是伪指令，`j Label` 相当于 `jal x0, Label`

0	1111101000	0	00000000	00000	1101111
imm[20]	imm[10:1]	imm[11]	imm[19:12]	rd	opcode

- PC = PC + offset （PC相对寻址）
- 访问相对PC，以**2字节**为单位的$\pm 2^{19}$范围内的地址空间/$\pm 2^{18}$ 字指令空间，或$\pm 2^{20}$字节（1MiB）地址范围

硬件/软件接口 大多数条件分支到达附近的位置，但偶尔会转移到很远的位置，远远超过条件分支指令中的 12 位地址能表示的范围。汇编程序解决该问题就如同处理大地址或常量的方法一样：插入无条件跳转到分支目标，并将条件取反，以便条件分支决定是否跳过该无条件跳转。

例题 | 远距离分支

寄存器 x10 等于 0 时给定一个分支，

```
beq   x10, x0, L1
```

用一对提供更大分支距离的指令替换它。

答案 | 用下面的指令替换短地址条件分支指令：

```
      bne     x10, x0, L2
      jal     x0, L1
L2:
```

CORE INSTRUCTION FORMATS

	31 27	26 25	24 20	19 15	14 12	11 7	6 0
R	funct7		rs2	rs1	funct3	rd	opcode
I	imm[11:0]			rs1	funct3	rd	opcode
S	imm[11:5]		rs2	rs1	funct3	imm[4:0]	opcode
SB	imm[12|10:5]		rs2	rs1	funct3	imm[4:1|11]	opcode
U	imm[31:12]					rd	opcode
UJ	imm[20|10:1|11|19:12]					rd	opcode

RISC-V寻址模式

立即数寻址

- 操作数是指令本身包含的常量

寄存器寻址

- 操作数在寄存器中

> addi x3, x4, 10：一条指令可以使用不止一种寻址方式

基址寻址（偏移寻址）

- 操作数于内存中，其地址是寄存器和指令中的常量之和
- 例如：`lw x10, 12(x15)`　基址寄存器(x15) + 偏移量(12)

• 对于以下C语句，编写相应的RISC-V汇编代码。

B [8] = A [i – j];　//long long int A[1024], B[1024];

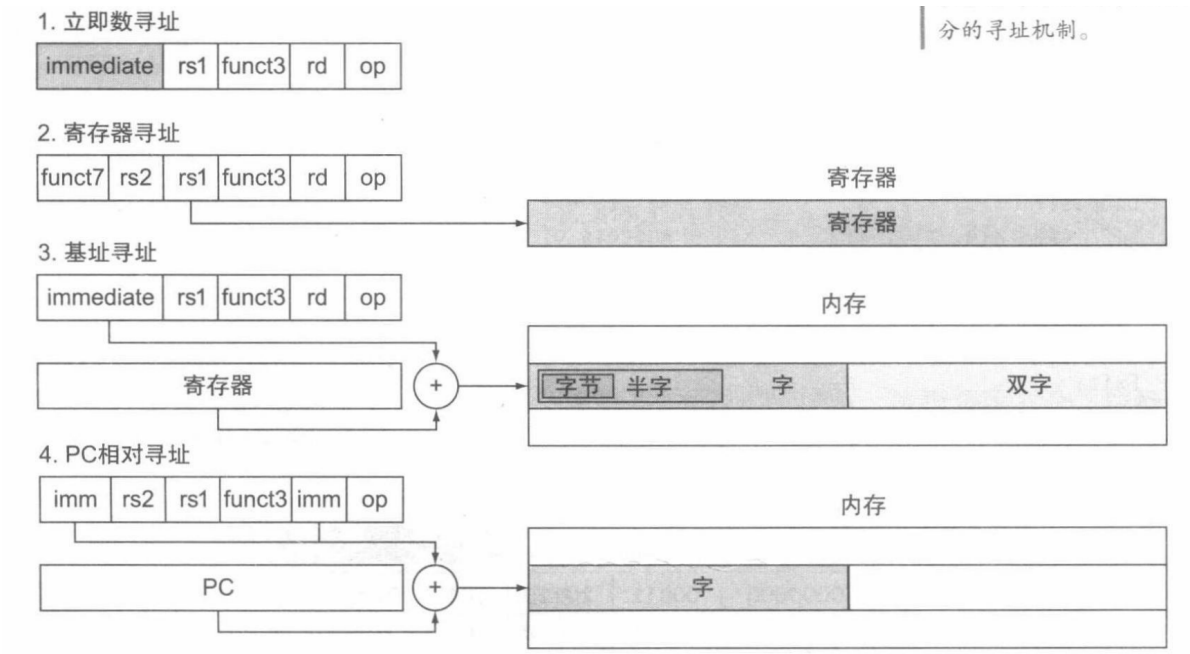
假设:为变量i和j分配寄存器x28和x29,A和B的基地址分别在寄存器x10和x11中。

```
sub x30, x28, x29       #i-j
slli x30, x30, 3        #8*(i-j)
add x30,x10,x30         #&A[i-j]--->x30
ld x30, 0(x30)          #A[i-j]--->x30    直接执行会出错
sd x30, 64(x11)         #x30--->B[8]
```

PC相对寻址

- 分支地址是PC与指令中立即数之和
- 例如：

```
Loop:
   beq x19, x10, End
   add x18, x18, x10
   addi x19, x19, -1
   jal x0, Loop
End:..
```



案例分析

数组与指针

用下标访问数组元素	用指针访问数组元素

clear1(long long int array[], int size){
int i;
for(i=0; i<size; i+=1)
array[i]=0;
} //a0中存array首地址,a1中存size
	clear2(long long int *array, int size){
	long long int *p;
	for(p=&array[0]; p< &array[size];p=p+1)
	***p**=0;
	} //a0中存array首地址,a1中存size

addi t0,zero,0 #i=0 **loop1**: **slli t1,t0,3** #t1=i*8 add t2,a0,t1 #t2=array[i]地址 sd zero,0(t2) #array[i]=0 addi t0,t0,1 #i=i+1元素计数加1 blt t0,a1,loop1 #i<size, 循环	addi t0, a0, 0 #t0=array[0]地址 **slli t1, a1,3** #t1=size*8 add t2,a0,t1 #t2=最后一个元素的地址 **loop2:**sd zero,0(t0) #memory[p]=0 addi t0,t0,8 #p=p+8 bltu t0,t2,loop2 #未到最后则循环

- 指针版本的循环体指令数少于数组版本
- 高级语言编程推荐使用下标方式访问数组元素

RISC-V冒泡排序

- (C语言)swap过程（叶过程：不调用其他过程的过程）

  RISC-V：假设v保存在a0，k保存在a1，temp保存在t0

swap 过程体的代码

swap 剩余的 C 代码如下所示：

```
temp   = v[k];
v[k]   = v[k+1];
v[k+1] = temp;
```

完整的 swap 过程

我们现在已经得到完整的例程。剩下的就是添加过程标签和返回跳转。

```
swap:
  slli   x6, x11, 3   // reg x6 = k * 8
  add    x6, x10, x6  // reg x6 = v + (k * 8)
  ld     x5, 0(x6)    // reg x5 (temp) = v[k]
  ld     x7, 8(x6)    // reg x7 = v[k + 1]
  sd     x7, 0(x6)    // v[k] = reg x7
  sd     x5, 8(x6)    // v[k+1] = reg x5 (temp)
  jalr   x0, 0(x1)    // return to calling routine
```

- C版本的sort过程（非叶过程）

  假设v保存在x10，n保存在x11，i保存在x19，j保存在x20

```
void sort (long long int v[], size_t int n)
{
    size_t i, j;
    for (i = 0; i < n; i += 1) {
        for (j = i - 1; j >= 0 && v[j] > v[j + 1]; j -= 1) {
            swap(v,j);
        }
    }
}
```

  外层循环的RV实现(for (i = 0; i <=n; i++)

```
    li x19,0 # i = 0
for1tst:
    bge x19,x11,exit1 # 如果i≥n，跳转到exit1
    .....#外层循环的函数体，包含内层循环
    addi x19,x19,1 # i += 1
    j for1tst # 跳转到外层循环的判断语句
exit1:
```

  内层循环的RV实现

```
//for (j = i − 1; j >= 0 && v[j] > v[j + 1]; j − = 1) {  ...}
    addi x20,x19,-1     # j = i −1
for2tst:
    blt x20,x0,exit2    # 如果j < 0，跳转到exit2
    slli x5,x20,3       # x5 = j * 8
    add x5,x10,x5       # x5 = v + (j * 8)
    ld x6,0(x5)         # x6 = v[j]
    ld x7,8(x5)         # x7 = v[j + 1]
    ble x6,x7,exit2     # 如果x6 ≤ x7，转exit2
    mv x21, x10         # 将参数x10复制到x21
    mv x22, x11         # 将参数x11复制到x22
    mv x10, x21         # swap的第一个参数是v
    mv x11, x20         # swap的第二个参数是j
    jal x1,swap         # 调用swap
    addi x20,x20,-1     # j –= 1
    j for2tst       #跳转到内层循环判断语句
exit2:...
```